

pytop Kullanım Kılavuzu

Nokta-Küme Topolojisi ve Metrik Uzaylar

Kadirhan Polat

2026

İçindekiler

Ön Söz

iv

I Nokta-Küme Topolojisi 1

1 Topolojik Uzaylar 2

1.1	Konu	2
1.1.1	Sezgisel Giriş	2
1.1.2	Formal Tanım	2
1.1.3	Temel Örnekler	2
1.1.4	Baz ve Alt-Baz	3
1.1.5	Sonlu ve Sonsuz Uzaylar	3
1.2	Teoremler	3
1.3	Algoritmalar	4
1.3.1	Bazdan Topoloji Üretimi	4
1.3.2	Alt-Bazdan Topoloji Üretimi	4
1.4	pytop API	4
1.4.1	Sınıflar	4
1.4.2	Oluşturucular	5
1.4.3	Tag Sistemi	5
1.5	Örnekler	5
1.5.1	Örnek 1: Sierpiński Uzayı	5
1.5.2	Örnek 2: Ayrık Topoloji	6
1.5.3	Örnek 3: <code>make_topology</code> ile Manuel İnşa	6
1.5.4	Örnek 4: Alexandrov Zincir Uzayı	6
1.5.5	Örnek 5: Bazdan Topoloji Üretimi	7
1.5.6	Örnek 6: Gerçek Doğru	7
1.6	Alıştırmalar	8

2 Yüklemler ve Altküme Operatörleri 9

2.1	Konu	9
2.1.1	Altküme Yüklemleri	9
2.1.2	Altküme Operatörleri	9
2.1.3	Komşuluk Sistemi	9
2.2	Teoremler	10
2.3	Algoritmalar	10
2.4	pytop API	11
2.5	Örnekler	11

2.5.1	Örnek 1: Kapanış ve İç — Sierpiński Uzayı	11
2.5.2	Örnek 2: Türev Kümesi	12
2.5.3	Örnek 3: Komşuluk Sistemi	12
2.6	Alıştırmalar	13
3	Ayrılma Aksiyomları	14
3.1	Konu	14
3.2	Teoremler	14
3.3	Algoritmalar	15
3.4	pytop API	15
3.5	Örnekler	15
3.5.1	Örnek 1: Sierpiński — T_0 ✓, T_1 ×	15
3.5.2	Örnek 2: Kosonlu $\mathbb{N}$ — T_1 ✓, T_2 ×	16
3.5.3	Örnek 3: Ayrılma Zinciri — Ayrık	16
3.6	Alıştırmalar	16
4	Kompaktlık	17
4.1	Konu	17
4.1.1	Kompaktlık Tanımı	17
4.1.2	Kompaktlık Varyantları	17
4.2	Teoremler	17
4.3	Algoritmalar	18
4.3.1	Sonlu Uzayda Kompaktlık	18
4.3.2	analyze_compactness_variants — Tag Tabanlı Çıkarım	18
4.4	pytop API	18
4.5	Örnekler	18
4.6	Alıştırmalar	19
5	Bağlantılılık	21
5.1	Konu	21
5.1.1	Bağlantılılık Kavramları	21
5.1.2	Clopen Ayrılma	21
5.2	Teoremler	21
5.3	Algoritmalar	22
5.3.1	Sonlu Bağlantılılık — $O(\tau ^2)$	22
5.4	pytop API	22
5.5	Örnekler	22
5.6	Alıştırmalar	23
6	Sayılabilirlik Aksiyomları	24
6.1	Konu	24
6.1.1	Sayılabilirlik Aksiyomları	24
6.2	Teoremler	24
6.3	Algoritmalar	25
6.3.1	Sonlu Uzayda Sayılabilirlik	25
6.3.2	Sonsuz Uzayda: Tag-Tabanlı Çıkarım	25
6.4	pytop API	25
6.5	Örnekler	25
6.6	Alıştırmalar	26

II	Metrik Uzaylar	27
7	Metrik Uzaylar	28
7.1	Konu	28
7.1.1	Metrik Aksiyomları	28
7.1.2	Temel Kavramlar	28
7.1.3	Standart Metrikler	28
7.2	Teoremler	29
7.3	Algoritmalar	29
7.3.1	validate_metric — $O(X ^3)$	29
7.3.2	open_ball — $O(X)$	29
7.4	pytop API	30
7.5	Örnekler	30
7.6	Alıştırmalar	31
8	Metrik Tamlık ve Kompaktlık	32
8.1	Konu	32
8.1.1	Cauchy Dizisi ve Tamlık	32
8.1.2	Tamamen Sınırlılık	32
8.1.3	Metrik Kompaktlık Karakterizasyonu	32
8.2	Teoremler	33
8.3	Algoritmalar	33
8.3.1	Sonlu Metrikte is_complete ve is_totally_bounded	33
8.3.2	metric_compactness_check	33
8.4	pytop API	33
8.5	Örnekler	33
8.6	Alıştırmalar	34
9	Metrik Fonksiyonlar ve Sözleşmeler	35
9.1	Konu	35
9.1.1	Metrik Fonksiyon Sınıfları	35
9.1.2	Lipschitz Sabiti	35
9.2	Teoremler	35
9.3	Algoritmalar	36
9.3.1	classify_finite_metric_map — $O(X ^2)$	36
9.4	pytop API	36
9.5	Örnekler	36
9.6	Alıştırmalar	37
	Dizin	38

Ön Söz

Bu kılavuz `pytop` paketinin nokta-küme topolojisi ve metrik uzaylar modüllerini kapsamlı biçimde tanıtmaktadır. Her bölüm aynı yapıyı izler:

1. Teorik giriş ve formal tanımlar
2. Temel teoremler (kanıt iskeletleriyle)
3. Algoritmalar (sözde kod ve karmaşıklık)
4. `pytop` API
5. Çalışan örnekler
6. Alıştırmalar

Örnekler aynı zamanda `docs/user_guide/python/` altındaki Python script'lerinde ve `docs/user_guide/notebook/` altındaki Jupyter defterlerinde çalıştırılabilir biçimde mevcuttur.

Kısım I

Nokta-Küme Topolojisi

Bölüm 1

Topolojik Uzaylar

1.1 Konu

1.1.1 Sezgisel Giriş

Topoloji, bir kümedeki noktaların birbirine “yakın” olup olmadığını, aralarındaki sürekliliği ve şekilsel özellikleri, mesafe kavramına gerek duymadan inceleyen matematiksel bir disiplindir. Bunu mümkün kılan temel araç *açık küme* kavramıdır.

Düşünelim: gerçek doğrudaki (a, b) aralığı “açık”tır; her noktasının çevresinde küçük bir açık aralık daha bulunur. Bu sezgiyi soyutlayarak *herhangi* bir küme üzerinde topoloji tanımlamak mümkündür.

1.1.2 Formal Tanım

Tanım 1.1 (Topolojik Uzay). Bir X kümesi ve $\tau \subseteq \mathcal{P}(X)$ ailesi verilsin. (X, τ) bir **topolojik uzay**, τ ise X üzerinde bir **topoloji** olarak adlandırılır, eğer aşağıdaki üç aksiyom sağlanıyorsa:

$$(T1) \quad \emptyset \in \tau \text{ ve } X \in \tau$$

$$(T2) \quad U, V \in \tau \Rightarrow U \cap V \in \tau$$

$$(T3) \quad \{U_\alpha\}_{\alpha \in I} \subseteq \tau \Rightarrow \bigcup_{\alpha \in I} U_\alpha \in \tau$$

τ 'nın elemanları **açık küme** olarak adlandırılır.

1.1.3 Temel Örnekler

Topoloji	Tanım	Açık kümeler
Ayrık (discrete)	Her alt küme açık	$\tau = \mathcal{P}(X)$
İndirgenmiş (indiscrete)	Yalnızca $\emptyset$ ve X açık	$\tau = \{\emptyset, X\}$
Kosonlu (cofinite)	U açık $\iff U = \emptyset$ veya $X \setminus U$ sonlu	—
Sierpiński	$X = \{0, 1\}$, $\tau = \{\emptyset, \{1\}, X\}$	3 açık küme

Tablo 1.1: Standart topoloji örnekleri

1.1.4 Baz ve Alt-Baz

Tanım 1.2 (Baz). $\mathcal{B} \subseteq \tau$ ailesi, her $U \in \tau$ ve her $x \in U$ için $x \in B \subseteq U$ sağlayan bir $B \in \mathcal{B}$ varsa τ 'ya **baz** denir.

Baz Koşulları:

$$(B1) \bigcup \mathcal{B} = X$$

$$(B2) B_1, B_2 \in \mathcal{B} \text{ ve } x \in B_1 \cap B_2 \text{ ise bir } B_3 \in \mathcal{B} \text{ vardır: } x \in B_3 \subseteq B_1 \cap B_2$$

Tanım 1.3 (Alt-Baz). $\mathcal{S} \subseteq \mathcal{P}(X)$ ailesi; sonlu kesişimleri baz, onların birleşimleri ise τ 'yu oluşturur.

1.1.5 Sonlu ve Sonsuz Uzaylar

pytop'ta iki temel sınıf bulunur:

- **FiniteTopologicalSpace:** Taşıyıcı (X) sonlu; topoloji açıkça **frozenset**'ler koleksiyonu olarak saklanır. Tüm hesaplamalar tam ve kesindir.
- **Sonsuz türevler:** Taşıyıcı simgesel ('R', 'N'); **topology =None**; etiket tabanlı sembolik çıkarım yapılır.

1.2 Teoremler

Teorem 1.4 (Aksiyomların Yeterliliği). (X, τ) bir topolojik uzay olsun. T1–T3 aksiyomları, sonlu herhangi bir açık küme ailesinin kesişiminin τ 'da yer aldığını garanti eder:

$$U_1, \dots, U_n \in \tau \Rightarrow \bigcap_{i=1}^n U_i \in \tau.$$

Not: Sonsuz kesişim gerekmez. Gerçek doğrudan $\bigcap_{n=1}^{\infty} (-\frac{1}{n}, \frac{1}{n}) = \{0\}$ açık değildir.

Teorem 1.5 (Baz Teoremi). $\mathcal{B} \subseteq \mathcal{P}(X)$ ailesi (B1)–(B2) koşullarını sağlıyorsa,

$$\tau_{\mathcal{B}} = \{U \subseteq X : \forall x \in U, \exists B \in \mathcal{B}, x \in B \subseteq U\}$$

bir topoloji olur ve $\mathcal{B}$ bu topolojiye baz oluşturur.

Kanıt İskeleti. (T1): $\emptyset \in \tau_{\mathcal{B}}$ boş koşul nedeniyle; $X \in \tau_{\mathcal{B}}$ (B1) nedeniyle. (T2): $U, V \in \tau_{\mathcal{B}}$ ve $x \in U \cap V$ için B_U, B_V bul; (B2) ile $B_3 \subseteq B_U \cap B_V$. (T3): birleşim noktası için doğrudan baz elemanı al. $\square$

Teorem 1.6 (Karşılaştırma). X üzerinde iki topoloji $\tau_1 \subseteq \tau_2$ ise τ_1 **kaba (coarser)**, τ_2 **ince (finer)** topoloji olarak adlandırılır. Ayırık topoloji en ince, indirgenmiş topoloji en kaba topolojidir.

Algorithm 1 Bazdan Topoloji Üretimi**Require:** X (taşıyıcı), $\mathcal{B}$ (baz ailesi)**Ensure:** τ (kapalı topoloji)

```

1:  $\tau \leftarrow \{\emptyset, X\}$ 
2: for each non-empty  $\mathcal{S} \subseteq \mathcal{B}$  do
3:    $\tau.add(\bigcup_{B \in \mathcal{S}} B)$ 
4: end for
5: CloseUnderIntersection( $\tau$ ) return  $\tau$ 

```

Algorithm 2 Kesişim Altında Kapatma

```

1:  $changed \leftarrow \mathbf{true}$ 
2: while  $changed$  do
3:    $changed \leftarrow \mathbf{false}$ 
4:   for each  $U, V \in \tau$  do
5:     if  $U \cap V \notin \tau$  then
6:        $\tau.add(U \cap V)$ ;  $changed \leftarrow \mathbf{true}$ 
7:     end if
8:   end for
9: end while

```

1.3 Algoritmalar

1.3.1 Bazdan Topoloji Üretimi

Problem: X ve $\mathcal{B} \subseteq \mathcal{P}(X)$ verilsin; $\tau_{\mathcal{B}}$ 'yi hesapla.**Karmaşıklık:**

- En kötü durum: $O(|\mathcal{B}| \cdot 2^{|\mathcal{B}|})$ (tüm alt-ailelerin birleşimi)
- Pratik (çakışmayan baz elemanları): $O(|\tau| \cdot |\mathcal{B}|)$
- Uzay: $O(|\tau|)$ açık küme saklar

1.3.2 Alt-Bazdan Topoloji Üretimi

 $\mathcal{S}$ verilsin; önce $\mathcal{S}$ 'nin sonlu kesişimlerinden baz oluştur, ardından Algoritma 1'yi uygula.**Karmaşıklık:** $O(|\mathcal{S}|^k \cdot 2^{|\mathcal{S}|^k})$ — k : kesişim derinliği.

1.4 pytop API

1.4.1 Sınıflar

Listing 1.1: Temel sınıf importları

```

1 from pytop import TopologicalSpace, FiniteTopologicalSpace

```

Alanlar: `carrier` (altta yatan küme), `topology` (açık kümeler), `tags` (özellik etiketleri), `metadata` (ek bilgiler).

Sınıf	Taşıyıcı	Topoloji
TopologicalSpace	Any	Any (None olabilir)
FiniteTopologicalSpace	sonlu frozenset	frozenset[frozenset]

Tablo 1.2: Temel uzay sınıfları

1.4.2 Oluşturucular

Listing 1.2: Topoloji oluşturucu importları

```

1 from pytop import (
2     make_topology, discrete_topology, indiscrete_topology,
3     cofinite_topology, sierpinski_space,
4     topology_from_basis, topology_from_subbasis,
5 )

```

Fonksiyon	İmza	Açıklama
make_topology	(carrier, *open_sets)	Açık küme listesiyle inşa
discrete_topology	(*elements)	Ayrık topoloji
indiscrete_topology	(*elements)	İndirgenmiş topoloji
cofinite_topology	(*elements)	Kosonlu topoloji
sierpinski_space	()	Sierpiński uzayı
topology_from_basis	(carrier, basis)	Bazdan topoloji üret
topology_from_subbasis	(carrier, subbasis)	Alt-bazdan topoloji üret

Tablo 1.3: Topoloji oluşturucu fonksiyonlar

1.4.3 Tag Sistemi

pytop nesneleri tags kümesi aracılığıyla topolojik özellikler taşır. Örnek etiketler: "finite", "discrete", "t0", "t1", "hausdorff", "compact", "connected", "metric".

1.5 Örnekler

1.5.1 Örnek 1: Sierpiński Uzayı

Listing 1.3: Sierpiński uzayı

```

1 from pytop import sierpinski_space
2
3 s = sierpinski_space()
4 print("Taşıyıcı:", s.carrier)
5 print("Topoloji:", sorted(str(t) for t in s.topology))
6 print("Etiketler:", sorted(s.tags))

```

Listing 1.4: Çıktı

```

1 Tasiyici: frozenset({0, 1})
2 Topoloji: ['frozenset()', 'frozenset({0, 1})', 'frozenset({1})']
3 Etiketler: ['compact', 'connected', 'finite', 't0']

```

Topoloji tam olarak üç açık küme içerir: $\emptyset$, $\{1\}$, $\{0,1\}$. $\{0\}$ açık değildir — bu T_0 fakat T_1 olmayan özelliğin kaynağıdır.

1.5.2 Örnek 2: Ayırık Topoloji

Listing 1.5: Ayırık topoloji

```

1 from pytop import discrete_topology
2
3 d = discrete_topology(1, 2, 3)
4 print("|tau| =", len(d.topology))
5 print("Etiketler:", sorted(d.tags))

```

Listing 1.6: Çıktı

```

1 |tau| = 8
2 Etiketler: ['discrete', 'finite', 'hausdorff', 'metrizable', 'normal',
3            , 'regular']

```

$n = 3$ için $|\tau| = 2^3 = 8$. Ayırık topoloji Hausdorff, regüler, normal, metriklenebilirdir.

1.5.3 Örnek 3: make_topology ile Manuel İnşa

Listing 1.7: Manuel topoloji inşası

```

1 from pytop import make_topology
2
3 sp = make_topology({1, 2, 3}, {1}, {2, 3})
4 print("|tau| =", len(sp.topology))
5 print("Acik kumeler:", sorted(str(t) for t in sp.topology))

```

Listing 1.8: Çıktı

```

1 |tau| = 4
2 Acik kumeler: ['frozenset()', 'frozenset({1, 2, 3})',
3              'frozenset({1})', 'frozenset({2, 3})']

```

$\{1\}$ ve $\{2,3\}$ açık verilince, $\emptyset$ ve X otomatik eklenir.

1.5.4 Örnek 4: Alexandrov Zincir Uzayı

Listing 1.9: Zincir uzayı

```

1 from pytop import finite_chain_space
2
3 c = finite_chain_space(3)
4 print("Tasiyici:", c.carrier)
5 print("Topoloji:", sorted(str(t) for t in c.topology))

```

Listing 1.10: Çıktı

```

1 Tasiyici: (1, 2, 3)
2 Topoloji: ['set()', '{1, 2, 3}', '{1, 2}', '{1}']

```

Her açık küme bir “önek”tir. Alexandrov topolojisinde noktalar soldan sağa giderek az açık hale gelir.

1.5.5 Örnek 5: Bazdan Topoloji Üretimi

Listing 1.11: Bazdan topoloji

```

1 from pytop import topology_from_basis
2
3 b = [{1}, {2, 3}, {4}]
4 ts = topology_from_basis({1, 2, 3, 4}, b)
5 print("Baz:", [set(x) for x in b])
6 print("|tau| =", len(ts.topology))
7 print("Topoloji:", sorted(str(t) for t in ts.topology))

```

Listing 1.12: Çıktı

```

1 Baz: [{1}, {2, 3}, {4}]
2 |tau| = 8
3 Topoloji: ['set()', '{1, 2, 3, 4}', '{1, 2, 3}', '{1, 4}',
4           '{1}', '{2, 3, 4}', '{2, 3}', '{4}']

```

Baz $\{\{1\}, \{2, 3\}, \{4\}\}$ bir bölüntüden oluşur; 8 açık küme tüm birleşim kombinasyonlarından elde edilir.

1.5.6 Örnek 6: Gerçek Doğru

Listing 1.13: Gerçek doğru

```

1 from pytop import real_line_metric
2
3 rl = real_line_metric()
4 print("Tasiyici:", rl.carrier)
5 print("Etiketler:", sorted(rl.tags))

```

Listing 1.14: Çıktı

```

1 Tasiyici: R
2 Etiketler: ['complete', 'connected', 'first_countable', 'hausdorff',
3           'infinite', 'lindelof', 'metric', 'not_compact',
4           'path_connected', 'second_countable', 'separable', 't0',
5           't1', 'uncountable']

```

Gerçek doğru simgesel temsil edilir; topolojik özellikler etiketlerde kodlanmıştır.

1.6 Alıştırmalar

Kodlama Alıştırmaları

- K1.** $X = \{a, b, c\}$ üzerinde `make_topology` kullanarak T1 ama T2 (Hausdorff) olmayan bir topoloji inşa edin. (İpucu: kosonlu topolojiyi düşünün.)
- K2.** `topology_from_subbasis({1,2,3,4}, [{1,2},{3,4},{2,3}])` çağrısının ürettiği topolojinin kaç açık küme içerdiğini bulun ve açık kümeleri listeleyin.
- K3.** `finite_chain_space(5)` ile bir Alexandrov-5 zinciri oluşturun; topolojinin kaç açık küme içerdiğini ve hangi elemanların “en açık” nokta olduğunu bulun.

Teori Alıştırmaları

- T1.** $X = \{1, 2, 3\}$ üzerinde kaç farklı topoloji tanımlanabilir? Bu sayıyı T1–T3 aksiyomlarını doğrudan kontrol ederek hesaplamak yerine neden baz teoremini kullanmak daha pratiktir?
- T2.** Ayrık topolojinin her zaman diğer topolojilerden daha ince, indirgenmiş topolojinin ise daha kaba olduğunu kanıtlayın. Kanıt için T1–T3 aksiyomlarından hangisi gereklidir?

Bölüm 2

Yüklemler ve Altküme Operatörleri

2.1 Konu

2.1.1 Altküme Yüklemeleri

(X, τ) bir topolojik uzay ve $A \subseteq X$ olsun.

Yüklem	Tanım
Açık (open)	$A \in \tau$
Kapalı (closed)	$X \setminus A \in \tau$
Clopen	$A \in \tau$ ve $X \setminus A \in \tau$
Yoğun (dense)	Her boş olmayan $U \in \tau$ için $U \cap A \neq \emptyset$
Hiçbir yerde-yoğun	$\text{int}(\overline{A}) = \emptyset$

Tablo 2.1: Altküme yüklemeleri

2.1.2 Altküme Operatörleri

Operatör	Gösterim	Tanım
Kapanış	$\overline{A} = \text{cl}(A)$	A 'yı içeren en küçük kapalı küme
İç	$A^\circ = \text{int}(A)$	A 'ya dahil en büyük açık küme
Sınır	$\partial A = \text{bd}(A)$	$\overline{A} \setminus A^\circ$
Türev kümesi	$A' = d(A)$	A 'nın birikme noktaları kümesi

Tablo 2.2: Altküme operatörleri

Birikme noktası: $x \in A' \iff$ her $U \ni x$ açık için $U \cap (A \setminus \{x\}) \neq \emptyset$.

2.1.3 Komşuluk Sistemi

Tanım 2.1 (Komşuluk Sistemi). $x \in X$ için:

$$N(x) = \{N \subseteq X : \exists U \in \tau, x \in U \subseteq N\}$$

Komşuluk Aksiyomları (N1–N4):

- (N1) $x \in N$, her $N \in N(x)$ için
 (N2) $X \in N(x)$
 (N3) $N_1, N_2 \in N(x) \Rightarrow N_1 \cap N_2 \in N(x)$
 (N4) $N \in N(x), N \subseteq M \Rightarrow M \in N(x)$

2.2 Teoremler

Teorem 2.2 (Kuratowski Kapanış Aksiyomları). $\text{cl} : \mathcal{P}(X) \rightarrow \mathcal{P}(X)$ operatörü şu dört özelliği sağlar:

- (K1) $\text{cl}(\emptyset) = \emptyset$
 (K2) $A \subseteq \text{cl}(A)$
 (K3) $\text{cl}(\text{cl}(A)) = \text{cl}(A)$
 (K4) $\text{cl}(A \cup B) = \text{cl}(A) \cup \text{cl}(B)$

Bu dört özelliği sağlayan her operatör bir topoloji tanımlar.

Teorem 2.3 (İç-Kapanış Dualitesi). Her $A \subseteq X$ için:

$$A^\circ = X \setminus \text{cl}(X \setminus A), \quad \text{cl}(A) = X \setminus \text{int}(X \setminus A).$$

Teorem 2.4 (Komşuluk Sistemi Round-Trip). $N1-N_4$ 'ü sağlayan her $\{N(x)\}_{x \in X}$ ailesi, $\tau = \{U : \forall x \in U, U \in N(x)\}$ şeklinde tek bir topolojiyi geri üretir.

2.3 Algoritmalar

Algorithm 3 Sonlu Kapanış Hesabı

Require: $A \subseteq X, \tau$

Ensure: $\text{cl}(A)$

- 1: $\text{closed} \leftarrow \{X \setminus U : U \in \tau\}$
 - 2: $\text{result} \leftarrow X$
 - 3: **for** each $C \in \text{closed}$ **do**
 - 4: **if** $A \subseteq C$ **and** $|C| < |\text{result}|$ **then**
 - 5: $\text{result} \leftarrow C$
 - 6: **end if**
 - 7: **end for** **return** result
-

Karmaşıklık: $O(|\tau| \cdot |X|)$.

Karmaşıklık: $O(|X| \cdot |\tau|)$.

Algorithm 4 Türev Kümesi Hesabı**Require:** $A \subseteq X, \tau$ **Ensure:** $d(A)$

```

1:  $acc \leftarrow \emptyset$ 
2: for each  $x \in X$  do
3:   if her açık  $U \ni x$  için  $U \cap (A \setminus \{x\}) \neq \emptyset$  then
4:      $acc.add(x)$ 
5:   end if
6: end for return  $acc$ 

```

2.4 pytop API

Listing 2.1: Subset operatörleri importları

```

1 from pytop import (
2     analyze_predicate,
3     closure_of_subset, interior_of_subset, boundary_of_subset,
4     derived_set_of_subset,
5     is_open_subset, is_closed_subset, is_dense_subset,
6     is_nowhere_dense_subset,
7     neighborhood_system, character_at_point,
8     analyze_neighborhood_system,
9 )

```

Fonksiyon	İmza	Açıklama
analyze_predicate	(space, predicate, subset)	Yüklem sorgula
closure_of_subset	(space, subset)	Kapanış
interior_of_subset	(space, subset)	İç
boundary_of_subset	(space, subset)	Sınır
derived_set_of_subset	(space, subset)	Türev kümesi
neighborhood_system	(carrier, topology, point)	$N(x)$
character_at_point	(carrier, topology, point)	$\chi(x)$
analyze_neighborhood_system	(carrier, topology, point =None)	Tam analiz

Tablo 2.3: Altküme operatör fonksiyonları

Not: neighborhood_system ve ilgili fonksiyonlar carrier (liste) ve topology (liste) alır. Space nesnesi için: list(space.carrier), list(space.topology).

2.5 Örnekler

2.5.1 Örnek 1: Kapanış ve İç — Sierpiński Uzayı

```

1 from pytop import sierpinski_space, closure_of_subset,
2     interior_of_subset, boundary_of_subset
3 s = sierpinski_space()

```



```

4 print("cl({0}) =", closure_of_subset(s, {0}).value)
5 print("cl({1}) =", closure_of_subset(s, {1}).value)
6 print("int({0}) =", interior_of_subset(s, {0}).value)
7 print("int({1}) =", interior_of_subset(s, {1}).value)
8 print("bd({1}) =", boundary_of_subset(s, {1}).value)

```

Listing 2.2: Çıktı

```

1 cl({0}) = frozenset({0})
2 cl({1}) = frozenset({0, 1})
3 int({0}) = frozenset()
4 int({1}) = frozenset({1})
5 bd({1}) = frozenset({0})

```

$\{0\}$ 'ın kapanışı kendisidir: $\{0\} = X \setminus \{1\}$ zaten kapalıdır. $\{1\}$ 'in kapanışı X 'tir: $\{1\}$ kapalı değildir.

2.5.2 Örnek 2: Türev Kümesi

```

1 from pytop import sierpinski_space, derived_set_of_subset,
  is_nowhere_dense_subset
2
3 s = sierpinski_space()
4 print("d({0}) =", derived_set_of_subset(s, {0}).value)
5 print("d({1}) =", derived_set_of_subset(s, {1}).value)
6 print("{0} nowhere dense? =", is_nowhere_dense_subset(s, {0}).status)

```

Listing 2.3: Çıktı

```

1 d({0}) = frozenset()
2 d({1}) = frozenset({0})
3 {0} nowhere dense? = true

```

$d(\{1\}) = \{0\}$: 0'ı içeren her $\{0, 1\}$, $\{1\} \setminus \{0\} = \{1\}$ ile kesişir.

2.5.3 Örnek 3: Komşuluk Sistemi

```

1 from pytop import sierpinski_space, neighborhood_system,
  character_at_point
2
3 s = sierpinski_space()
4 carrier = list(s.carrier)
5 topology = list(s.topology)
6 print("N(0) =", neighborhood_system(carrier, topology, 0).value)
7 print("N(1) =", neighborhood_system(carrier, topology, 1).value)
8 print("chi(0) =", character_at_point(carrier, topology, 0).value)
9 print("chi(1) =", character_at_point(carrier, topology, 1).value)

```

Listing 2.4: Çıktı

```

1 N(0) = [['0', '1']]
2 N(1) = [['1'], ['0', '1']]

```

```

3 | chi(0) = 1
4 | chi(1) = 2

```

$N(0) = \{X\}$: 0 yalnızca X 'te açık. $N(1) = \{\{1\}, X\}$: yerel baz büyüklükleri $\chi(0) = 1$, $\chi(1) = 2$.

2.6 Alıştırmalar

Kodlama Alıştırmaları

- K1.** İndirgenmiş iki-noktalı topolojide her noktanın komşuluk sistemini hesaplayın.
- K2.** $X = \{1, 2, 3, 4\}$, $\tau = \{\emptyset, \{1, 2\}, \{3, 4\}, X\}$ topolojisinde $\text{bd}(\{1, 2, 3\})$ ve $\text{cl}(\{1, 2, 3\})$ hesaplayın.
- K3.** `finite_chain_space(4)` zincirinde $\{1, 2\}$ 'nin iç, kapanış ve sınır kümelerini bulun.

Teori Alıştırmaları

- T1.** $A^\circ = X \setminus \text{cl}(X \setminus A)$ eşitliğini Kuratowski aksiyomlarını kullanarak kanıtlayın.
- T2.** A 'nın yoğun olması için gerek ve yeter koşulun $\text{cl}(A) = X$ olduğunu gösterin.

Bölüm 3

Ayrılma Aksiyomları

3.1 Konu

Ayrılma aksiyomları, bir topolojik uzaydaki noktaların ve kapalı kümelerin birbirinden açık kümeler aracılığıyla ne ölçüde “ayrılabilirliğini” ölçer.

Aksiyom	İsim	Koşul
T0	Kolmogorov	$\forall x \neq y: \exists U \in \tau$ ayıran
T1	Fréchet	$\forall x \neq y$: iki yönlü ayırım
T2	Hausdorff	$\forall x \neq y$: disjoint $U \ni x, V \ni y$
T2.5	Urysohn	disjoint kapanışlı komşuluklar
T3	Regüler	T1 + nokta/kapalı ayırma
T3.5	Tychonoff	T1 + sürekli fonksiyon ayırma
T4	Normal	T1 + disjoint kapalılar ayırma

Tablo 3.1: Ayrılma aksiyomları

Sıralama: $T4 \Rightarrow T3.5 \Rightarrow T3 \Rightarrow T2.5 \Rightarrow T2 \Rightarrow T1 \Rightarrow T0$.

3.2 Teoremler

Teorem 3.1 (Ayrılma Zinciri). $T4 \Rightarrow T3.5 \Rightarrow T3 \Rightarrow T2.5 \Rightarrow T2 \Rightarrow T1 \Rightarrow T0$. Tersini genel olarak doğru değildir.

Teorem 3.2 (Urysohn Fonksiyon Teoremi). X normal ise ve C, D disjoint kapalı kümeler ise, $f : X \rightarrow [0, 1]$ sürekli vardır: $f|_C \equiv 0, f|_D \equiv 1$.

Teorem 3.3 (Tietze Genişleme). X normal ise her kapalı $A \subseteq X$ üzerinde $f : A \rightarrow [a, b]$ süreklisi tüm X 'e sürekli genişletilebilir.

Teorem 3.4 (Tychonoff Karakterizasyonu). $X, T3.5$ 'tir $\iff X$ bir küp $[0, 1]^I$ 'ye homeomorf gömülebilir.

Teorem 3.5 (Sonlu T1 $\iff$ Ayrık). Sonlu bir uzayda $T1 \iff$ ayrık topoloji.

İskelet. $T1 \Rightarrow$ her tekil küme kapalı $\Rightarrow$ her küme kapalı (sonlu birleşim) $\Rightarrow$ her küme açık. $\square$

3.3 Algoritmalar

Algorithm 5 Sonlu T0 Karar Prosedürü

```

1: for each pair  $(x, y)$  with  $x \neq y$  do
2:   if  $\nexists U \in \tau: (x \in U \wedge y \notin U) \text{ or } (y \in U \wedge x \notin U)$  then return false
3:   end if
4: end for return true

```

Karmaşıklık T0: $O(|X|^2 \cdot |\tau|)$. **Karmaşıklık T2:** $O(|X|^2 \cdot |\tau|^2)$.

3.4 pytop API

Listing 3.1: Ayrılma aksiyom importları

```

1 from pytop import (
2     is_t0, is_t1, is_t2, is_t2_5, is_t3, is_t4,
3     is_hausdorff, is_regular, is_normal, is_perfectly_normal,
4     separation_chain, analyze_separation,
5 )

```

Fonksiyon	İmza	Açıklama
is_t0 ...is_t4	(space)	Aksiyom kontrolü
separation_chain	(space)	Tüm zincir: dict[str, Result]
analyze_separation	(space, property = 'hausdorff')	Tek aksiyom sorgu

Tablo 3.2: Ayrılma fonksiyonları

3.5 Örnekler

3.5.1 Örnek 1: Sierpiński — T0 ✓, T1 ×

```

1 from pytop import sierpinski_space, is_t0, is_t1, is_t2
2 s = sierpinski_space()
3 print("T0:", is_t0(s).status)
4 print("T1:", is_t1(s).status)
5 print("T2:", is_t2(s).status)

```

Listing 3.2: Çıktı

```

1 T0: true
2 T1: false
3 T2: false

```

3.5.2 Örnek 2: Kosonlu $\mathbb{N}$ — $T_1 \checkmark$, $T_2 \times$

```

1 from pytop import naturals_cofinite, is_t1, is_t2
2 nc = naturals_cofinite()
3 print("T1:", is_t1(nc).status)
4 print("T2:", is_t2(nc).status)

```

Listing 3.3: Çıktı

```

1 T1: true
2 T2: false

```

3.5.3 Örnek 3: Ayrılma Zinciri — Ayrık

```

1 from pytop import discrete_topology, separation_chain
2 d = discrete_topology(1, 2, 3)
3 for prop, result in separation_chain(d).items():
4     print(f"    {prop:20s}: {result.status}")

```

Listing 3.4: Çıktı (kısa)

```

1     t0                : true
2     t1                : true
3     hausdorff         : true
4     t4                : true
5     perfectly_normal   : true

```

3.6 Alıştırmalar

Kodlama

K1. `make_topology({1,2,3},{1},{2},{1,2})` için `separation_chain` çalıştırın.

K2. `finite_chain_space(3)` üzerinde en yüksek sağlanan aksiyomu bulun.

K3. `two_point_indiscrete_space()` üzerinde T_0 , T_1 , T_2 test edin.

Teori

T1. $T_2 \Rightarrow T_1 \Rightarrow T_0$ implicasyonlarını kanıtlayın.

T2. Sonlu uzayda $T_1 \iff$ ayrık topoloji olduğunu gösterin.

Bölüm 4

Kompaktlık

Kompaktlık, sonsuz yapılarda “sınırlılık” fikrinin topolojik soyutlamasıdır. Açık örtü tanımını temel alınarak incelenmekte; varyantlar ve lokal kompaktlık da ele alınmaktadır.

4.1 Konu

4.1.1 Kompaktlık Tanımı

Tanım 4.1 (Kompakt Uzak). (X, τ) topolojik uzayı **kompakt** ise: Her açık örtü $\{U_\alpha\}$ için sonlu alt-örtü vardır:

$$X \subseteq \bigcup_{\alpha} U_{\alpha} \implies \exists \text{ sonlu } I : X \subseteq \bigcup_{\alpha \in I} U_{\alpha}.$$

4.1.2 Kompaktlık Varyantları

Varyant	Tanım
Kompakt	Her açık örtünün sonlu alt-örtüsü
Sayılabılır kompakt	Her sayılabılır örtünün sonlu alt-örtüsü
Ardışıklı kompakt	Her dizi yakınsak alt-dizi içerir
Sözde kompakt	Her sürekli $f : X \rightarrow \mathbb{R}$ sınırlıdır
Lindelöf	Her açık örtünün sayılabılır alt-örtüsü
σ -kompakt	Sayılabılır kompakt kümelerin birleşimi
Lokal kompakt	Her noktanın kompakt komşuluğu var

Tablo 4.1: Kompaktlık varyantları

Sıralama: Kompakt $\Rightarrow$ Sayılabılır kompakt $\Rightarrow$ Ardışıklı kompakt $\Rightarrow$ Lindelöf.

4.2 Teoremler

Teorem 4.2 (Heine-Borel). $\mathbb{R}^n$ ’de kompakt $\Leftrightarrow$ kapalı ve sınırlı.

Teorem 4.3. Kompakt + Hausdorff $\Rightarrow$ Normal (T_4).

Teorem 4.4. Kompakt uzaydan Hausdorff uzaya sürekli bijeksiyon $\Rightarrow$ homeomorfizma.

Teorem 4.5 (Tychonoff). *Keyfi kompakt uzayların kartezyen çarpımı kompaktır.*

Teorem 4.6 (Alexandroff Tek-Nokta Kompaktifikasyonu). *Her lokal kompakt Hausdorff uzay X için $X^* = X \cup \{\infty\}$ kompakt Hausdorff'tur.*

4.3 Algoritmalar

4.3.1 Sonlu Uzayda Kompaktlık

Sonlu uzay her zaman kompaktır: her örtü zaten sonlu. Algoritma: $O(1)$.

4.3.2 analyze_compactness_variants — Tag Tabanlı Çıkarım

Algorithm 6 AnalyzeVariants(X, τ)

```

1: if  $X$  sonlu then
2:   tüm varyantlar  $\leftarrow$  True; return VariantProfile(...)
3: end if
4: if 'compact'  $\in$  tags then
5:   countably_compact, sequentially_compact, lindelof  $\leftarrow$  True
6: end if
7: if 'metric'  $\wedge$  'compact'  $\in$  tags then
8:   sequentially_compact  $\leftarrow$  True
9: end if
```

4.4 pytop API

```

1 from pytop import (
2     is_compact,
3     is_lindelof,
4     is_locally_compact,
5 )
6 from pytop.compactness_variants import (
7     is_countably_compact,
8     is_sequentially_compact,
9     analyze_compactness_variants,
10 )
```

`analyze_compactness_variants(space)  $\rightarrow$  Result; .value bir dict içerir.`

4.5 Örnekler

Örnek 5.1 — Sonlu Uzaylar: Otomatik Kompakt

```

1 from pytop import sierpinski_space, discrete_topology,
   finite_chain_space, is_compact
2
```

```

3 s = sierpinski_space()
4 print("Sierpinski compact?", is_compact(s).status)
5 d = discrete_topology(1, 2, 3)
6 print("Discrete(3) compact?", is_compact(d).status)
7 c = finite_chain_space(3)
8 print("Chain(3) compact?", is_compact(c).status)

```

```

Sierpinski compact?      true
Chain(3) compact?        true
Discrete(3) compact?     true

```

Örnek 5.2 — Gerçek Doğru $\mathbb{R}$: Kompakt Değil

```

1 rl = real_line_metric()
2 print("compact?", is_compact(rl).status)
3 print("lindelof?", is_lindelof(rl).status)
4 print("locally_compact?", is_locally_compact(rl).status)

```

```

compact?      false
lindelof?     true
locally_compact? conditional

```

Örnek 5.3 — analyze_compactness_variants: Sierpiński

```

1 r = analyze_compactness_variants(s)
2 for key, val in r.value.items():
3     if hasattr(val, 'status'):
4         print(f"  {key:<25}: {val.status}")

```

```

representation      : finite
countably_compact    : true
sequentially_compact : true
pseudocompact        : true
lindelof             : true

```

4.6 Alıştırmalar

Kodlama

K1. `finite_chain_space(5)` ve `discrete_topology(1,2,3,4,5)` için `is_compact` karşılaştırın.

K2. `naturals_cofinite()` için `is_compact`, `is_lindelof`, `is_countably_compact` hesaplayın.

K3. `closed_unit_interval_metric()` ile `real_line_metric()` için `analyze_compactness_variants` çalıştırıp karşılaştırın.

Teori

- T1. Kompakt + sürekli $\Rightarrow$ görüntü kompakt olduğunu ispatlayın.
- T2. Heine-Borel teoreminin neden geçerli olduğunu: $[0, 1]$ neden kompakt, $(0, 1)$ neden kompakt değil?

Bölüm 5

Bağlantılılık

Bağlantılılık, bir topolojik uzayın iki ayrı açık parçaya bölünememesi özelliğidir. Yol-bağlantılılık daha güçlü bir kavramdır ve sürekli yolların varlığına dayanır.

5.1 Konu

5.1.1 Bağlantılılık Kavramları

Tanım 5.1 (Bağlantılı Uzay). (X, τ) uzayı **bağlantılı** ise: $X = U \cup V$, $U \cap V = \emptyset$, U, V açık $\Rightarrow U = \emptyset$ veya $V = \emptyset$.

Tanım 5.2 (Yol-Bağlantılı). $\forall x, y \in X: \exists$ sürekli $f : [0, 1] \rightarrow X$, $f(0) = x$, $f(1) = y$.

Kavram	Özellik
Bağlantılı	Clopen bölme yok
Yol-bağlantılı	Her iki nokta arasında sürekli yol var
Ark-bağlantılı	Yol-bağlantılı; yollar enjektif
Lokal bağlantılı	Her noktanın bağlantılı komşuluğu
Tamamen bağlantısız	Her bağlantılı alt küme tek noktadır

Tablo 5.1: Bağlantılılık hiyerarşisi

Sıralama: Ark-bağlantılı $\Rightarrow$ Yol-bağlantılı $\Rightarrow$ Bağlantılı.

5.1.2 Clopen Ayrılma

X bağlantısız $\Leftrightarrow$ boş olmayan trivial olmayan bir clopen $A \subsetneq X$ vardır.

5.2 Teoremler

Teorem 5.3. *Bağlantılı küme süreklilik altında bağlantılıdır: $f : X \rightarrow Y$ sürekli, X bağlantılı $\Rightarrow f(X)$ bağlantılı.*

Teorem 5.4. *Yol-bağlantılı $\Rightarrow$ bağlantılı. Tersisi genel olarak yanlış. (Karşı örnek: Topolojist sinüs eğrisi.)*

Teorem 5.5 (Ara Değer Teoremi). *$f : X \rightarrow \mathbb{R}$ sürekli, X bağlantılı $\Rightarrow f(X)$ bir aralıktır.*

Teorem 5.6. *Gerçek doğru $\mathbb{R}$ 'de bağlantılı alt kümeler tam olarak aralıklardır.*

5.3 Algoritmalar

5.3.1 Sonlu Bağlantılılık — $O(|\tau|^2)$

Algorithm 7 $\text{BagliMi}(X, \tau)$

```

1: for each non-empty  $A \subsetneq X$  do
2:   if  $A \in \tau$  and  $X \setminus A \in \tau$  then
3:     return False ▷ Clopen bölme bulundu
4:   end if
5: end for
6: return True

```

Karmaşıklık: $O(|\tau|^2)$ — clopen küme taraması.

5.4 pytop API

```

1 from pytop import (
2     is_connected,
3     is_path_connected,
4     is_locally_connected,
5     is_totally_disconnected,
6 )

```

5.5 Örnekler

Örnek 5.1 — Sierpiński: Bağlantılı

```

1 s = sierpinski_space()
2 print("connected?", is_connected(s).status)
3 print("path_connected?", is_path_connected(s).status)

```

```

connected?      true
path_connected? unknown

```

Örnek 5.2 — Ayrik Topoloji: Tamamen Bağlantısız

```

1 d = discrete_topology(1, 2, 3)
2 print("connected?", is_connected(d).status)
3 print("totally_disconn?", is_totally_disconnected(d).status)

```

```

connected?      false
totally_disconn? true

```

Örnek 5.3 — Gerçek Doğru ve $[0,1]$

```
1 r1 = real_line_metric()
2 ui = closed_unit_interval_metric()
3 print("R connected?", is_connected(r1).status)
4 print("[0,1] path_connected?", is_path_connected(ui).status)
```

```
R connected?          true
[0,1] path_connected? true
```

5.6 Alıştırmalar

Kodlama

- K1. `make_topology({1,2,3},{1},{2,3})` için `is_connected` hesaplayın.
- K2. `finite_chain_space(4)` zincirinin bağlantılı olup olmadığını kontrol edin.
- K3. İki noktalı ayrık ve indiscrete uzaylar üzerinde `is_connected`, `is_path_connected` karşılaştırın.

Teori

- T1. Bağlantılı + sürekli $\Rightarrow$ görüntü bağlantılı teoremini ispatlayın.
- T2. Yol-bağlantılı $\Rightarrow$ bağlantılı implicasyonunu ispatlayın.

Bölüm 6

Sayılabilirlik Aksiyomları

Sayılabilirlik aksiyomları, bir topolojik uzaydaki “bilgi”nin sayılabilir büyüklükte yapılarla tanımlanıp tanımlanamayacağını araştırır.

6.1 Konu

6.1.1 Sayılabilirlik Aksiyomları

Aksiyom	Tanım
Birinci sayılabilir	Her $x \in X$ için sayılabilir yerel baz
İkinci sayılabilir	Tüm topoloji için sayılabilir global baz
Ayrılabilir	Sayılabilir yoğun alt küme: $\overline{D} = X$
Lindelöf	Her açık örtünün sayılabilir alt-örtüsü

Tablo 6.1: Sayılabilirlik aksiyomları

Sıralamalar:

- 2^{nd} sayılabilir $\Rightarrow$ 1^{st} sayılabilir
- 2^{nd} sayılabilir $\Rightarrow$ ayrılabilir $\wedge$ Lindelöf
- Metrik $\Rightarrow$ 1^{st} sayılabilir
- Ayrılabilir metrik $\Rightarrow$ 2^{nd} sayılabilir

6.2 Teoremler

Teorem 6.1. 2^{nd} sayılabilir $\Rightarrow$ ayrılabilir $\wedge$ Lindelöf.

Teorem 6.2. Metrik uzay $\Rightarrow$ 1^{st} sayılabilir.

Kanıt İskeleti. Her x için $\{B(x, 1/n) : n \in \mathbb{N}\}$ sayılabilir yerel bazdır. □

Teorem 6.3. Ayrılabilir + metrik $\Rightarrow$ 2^{nd} sayılabilir.

Kanıt İskeleti. $D \subseteq X$ yoğun sayılabilir; $\{B(d, 1/n) : d \in D, n \in \mathbb{N}\}$ sayılabilir bazdır. □

Teorem 6.4 (Sorgenfrey Karşı Örneği). *Sorgenfrey doğrusu ayrılabilir ve Lindelöf'tür ama 2^{nd} sayılabilir değildir.*

6.3 Algoritmalar

6.3.1 Sonlu Uzayda Sayılabilirlik

Sonlu uzayda her aksiom trivially sağlanır. $O(1)$.

6.3.2 Sonsuz Uzayda: Tag-Tabanlı Çıkarım

pytop sembolik uzaylarda tag koridorlarından sayılabilirlik çıkarır:

- 'second_countable' $\in$ tags $\Rightarrow$ tüm aksiyomlar
- 'metric' $\wedge$ 'separable' $\Rightarrow$ 'second_countable'

6.4 pytop API

```
1 from pytop import (
2     is_first_countable,
3     is_second_countable,
4     is_separable,
5     is_lindelof,
6 )
```

6.5 Örnekler

Örnek 5.1 — Gerçek Doğru: Tüm 4 Aksiom

```
1 rl = real_line_metric()
2 print("1st countable?", is_first_countable(rl).status)
3 print("2nd countable?", is_second_countable(rl).status)
4 print("separable?", is_separable(rl).status)
5 print("lindehof?", is_lindelof(rl).status)
```

```
1st countable? true
2nd countable? true
separable?     true
lindehof?      true
```

Örnek 5.5 — Sorgenfrey Doğrusu: 2nd Countable $\times$

```
1 from pytop import sorgenfrey_line_like
2 sl = sorgenfrey_line_like()
3 print("1st countable?", is_first_countable(sl).status)
4 print("2nd countable?", is_second_countable(sl).status)
5 print("separable?", is_separable(sl).status)
```

```
1st countable?    true
2nd countable?    false
separable?        true
```

6.6 Alıştırmalar

Kodlama

- K1. `finite_chain_space(5)` üzerinde `is_first_countable` ve `is_second_countable` hesaplayın.
- K2. İki noktalı ayrık ve indiscrete uzaylar üzerinde `is_separable` karşılaştırın.
- K3. `integers_discrete()` için 2^{nd} sayılabilir mi?

Teori

- T1. 2^{nd} sayılabilir $\Rightarrow$ ayrılabilir olduğunu ispatlayın.
- T2. Sorgenfrey doğrusunun 2^{nd} sayılabilir olmadığını gösterin.

Kısım II

Metrik Uzaylar

Bölüm 7

Metrik Uzaylar

Metrik uzaylar, mesafe fonksiyonuna dayanan topolojik yapılardır. Her metrik uzay doğal olarak bir topolojik uzay oluşturur; bu yapı “indüklenen topoloji” olarak adlandırılır.

7.1 Konu

7.1.1 Metrik Aksiyomları

Tanım 7.1 (Metrik Uzay). M kümesi ve $d : M \times M \rightarrow [0, \infty)$ verilsin. (M, d) bir **metrik uzay**, d ise **metrik** ise:

(M1) **Özdeşlik**: $d(x, y) = 0 \Leftrightarrow x = y$

(M2) **Simetri**: $d(x, y) = d(y, x)$

(M3) **Üçgen**: $d(x, z) \leq d(x, y) + d(y, z)$

7.1.2 Temel Kavramlar

- **Açık top**: $B(x, r) = \{y \in M : d(x, y) < r\}$
- **Kapalı top**: $\bar{B}(x, r) = \{y \in M : d(x, y) \leq r\}$
- **İndüklenen topoloji**: $\tau_d = \{U : \forall x \in U, \exists \varepsilon > 0, B(x, \varepsilon) \subseteq U\}$
- **Çap**: $\text{diam}(A) = \sup\{d(x, y) : x, y \in A\}$

7.1.3 Standart Metrikler

Metrik	Tanım	Uzay
Öklid	$d(x, y) = x - y $	$\mathbb{R}$
Ayrık	$d(x, y) = 0$ veya 1	Herhangi X
ℓ^∞ (max)	$\max_i d_i(x_i, y_i)$	Çarpım uzayı
Sınırlı (capped)	$d'(x, y) = \min(d(x, y), 1)$	Eşdeğer topoloji

Tablo 7.1: Standart metrikler

7.2 Teoremler

Teorem 7.2. Her metrik uzay Hausdorff (T_2) ve 1^{st} sayılabilirdir.

Kanıt İskeleti. $B(x, d(x, y)/2)$ ve $B(y, d(x, y)/2)$ ayrık komşuluklardır; $\{B(x, 1/n)\}$ yerel bazdır. $\square$

Teorem 7.3 (Sınırlı Metrik Eşdeğerliği). $d'(x, y) = \min(d(x, y), 1)$ ile d aynı topolojiyi indükler.

Teorem 7.4. $\mathbb{R}^n$ üzerindeki $\max$, sum ve Öklid metrikleri topolojik olarak eşdeğerdir.

7.3 Algoritmalar

7.3.1 validate_metric — $O(|X|^3)$

Algorithm 8 ValidateMetric(X, d)

```

1: for each  $x \in X$  do
2:   if  $d(x, x) \neq 0$  then return False
3:   end if
4:   for each  $y \neq x$  do
5:     if  $d(x, y) = 0$  then return False
6:     end if
7:   end for
8: end for
9: for each  $x, y \in X$  do
10:  if  $d(x, y) \neq d(y, x)$  then return False
11:  end if
12: end for
13: for each  $x, y, z \in X$  do
14:  if  $d(x, z) > d(x, y) + d(y, z)$  then return False
15:  end if
16: end for
17: return True

```

Karmaşıklık: $O(|X|^3)$ — üçgen eşitsizliği dominates.

7.3.2 open_ball — $O(|X|)$

Algorithm 9 OpenBall(x, r, X, d)

```

1: return  $\{y \in X : d(x, y) < r\}$ 

```

7.4 pytop API

```

1 from pytop.metric_spaces import (
2     FiniteMetricSpace,
3     SymbolicMetricSpace,
4     validate_metric,
5 )
6 from pytop import (
7     open_ball,
8     closed_ball,
9     distance_to_subset,
10    diameter_of_subset,
11    is_bounded_subset,
12    capped_metric,
13 )

```

`FiniteMetricSpace(carrier =tuple, distance =dict)` — mesafe sözlüğü (a,b): float.

`open_ball(fms, point, radius)` → set döner (Result değil).

`capped_metric(distance_fn, cap =1.0)` → Callable döner.

7.5 Örnekler

Örnek 5.1 — Ayırık Metrik Uzay

```

1 points = [1, 2, 3, 4]
2 dist_discrete = {(a, b): (0 if a == b else 1)
3                     for a in points for b in points}
4 fms = FiniteMetricSpace(carrier=tuple(points), distance=dist_discrete
5 )
6 print("carrier:", fms.carrier)
7 print("d(1,2):", fms.distance[(1,2)])

```

carrier: (1, 2, 3, 4)

d(1,2): 1

Örnek 5.3 — open_ball

```

1 print("B(1, 0.5) =", open_ball(fms, 1, 0.5))
2 print("B(1, 1.5) =", open_ball(fms, 1, 1.5))

```

B(1, 0.5) = {1}

B(1, 1.5) = {1, 2, 3, 4}

Örnek 5.6 — capped_metric

```
1 cap_fn = capped_metric(fms_line.distance, cap=1.0)
2 print("d(0,3) capped:", cap_fn(0, 3))
3 print("d(0,1) capped:", cap_fn(0, 1))
```

d(0,3) capped: 1.0

d(0,1) capped: 1.0

7.6 Alıştırmalar

Kodlama

- K1. $\{A, B, C, D\}$ üzerinde bir metrik tanımlayın ve `validate_metric` ile doğrulayın. Üçgen eşitsizliğini ihlal eden bir örnek de deneyin.
- K2. Öklid metrikli $\{0, \dots, 9\}$ üzerinde $B(5, 3.0)$ ve $\bar{B}(5, 3.0)$ hesaplayın.
- K3. `normalized_metric` kullanarak bir metriği $[0, 1]$ 'e normalleştirin.

Teori

- T1. Her metrik uzayın Hausdorff olduğunu ispatlayın.
- T2. $d' = \min(d, 1)$ ve d aynı topolojiyi indüklediğini gösterin.

Bölüm 8

Metrik Tamlık ve Kompaktlık

Metrik tamlık (completeness), bir metrik uzayda Cauchy dizilerinin yakınsayıp yakınsamadığını araştıran temel bir kavramdır.

8.1 Konu

8.1.1 Cauchy Dizisi ve Tamlık

Tanım 8.1 (Cauchy Dizisi). (M, d) metrik uzayında $\{x_n\}$ dizisi **Cauchy** ise:

$$\forall \varepsilon > 0, \exists N \in \mathbb{N} : m, n \geq N \Rightarrow d(x_m, x_n) < \varepsilon.$$

Tanım 8.2 (Tam Metrik Uzay). (M, d) **tam** ise: Her Cauchy dizisi M 'de bir limite yakınsır.

8.1.2 Tamamen Sınırlılık

$A \subseteq M$ **tamamen sınırlı** ise: her $\varepsilon > 0$ için A 'nın sonlu bir ε -net kümesi vardır.

$$S = \{s_1, \dots, s_n\} \subseteq M, \quad \forall x \in A \exists i : d(x, s_i) < \varepsilon.$$

8.1.3 Metrik Kompaktlık Karakterizasyonu

$$(M, d) \text{ tam} \wedge \text{tamamen sınırlı} \iff (M, d) \text{ kompakt}.$$

Uzay	Tam?	Tam. sınırlı?	Kompakt?
$\mathbb{R}$	✓	×	×
$[0, 1]$	✓	✓	✓
$(0, 1)$	×	✓	×
$\mathbb{Q}$	×	×	×
Sonlu metrik	✓	✓	✓

Tablo 8.1: Tamlık ve kompaktlık örnekleri

8.2 Teoremler

Teorem 8.3 (Heine-Borel Metrik Genelleştirmesi). *Metrik uzayda kompaktlık $\Leftrightarrow$ tamlık $\wedge$ tamamen sınırlılık.*

Teorem 8.4 (Banach Sabit-Nokta Teoremi). *(M, d) tam metrik uzay; $T : M \rightarrow M$ büzülme ($K < 1$) $\Rightarrow T$ 'nin eşsiz sabit noktası x^* vardır: $T(x^*) = x^*$.*

Teorem 8.5 (Baire Kategorisi Teoremi). *Tam metrik uzay 1. kategoriden değildir.*

8.3 Algoritmalar

8.3.1 Sonlu Metrikte is__complete ve is__totally__bounded

Sonlu metrik uzayda her Cauchy dizisi eninde sonunda sabit: trivially tam. Taşıyıcı kendisi ε -net: trivially tamamen sınırlı. Her ikisi: $O(1)$.

8.3.2 metric_compactness_check

Algorithm 10 MetricCompactnessCheck(M, d)

```

1: complete  $\leftarrow$  is_complete( $M, d$ )
2: totally_bounded  $\leftarrow$  is_totally_bounded( $M, d$ )
3: return complete  $\wedge$  totally_bounded

```

8.4 pytop API

```

1 from pytop.metric_completeness import (
2     is_complete,
3     is_totally_bounded,
4     metric_compactness_check,
5     analyze_metric_completeness,
6 )

```

8.5 Örnekler

Örnek 5.1 — Sonlu Metrik: Tam ve Kompakt

```

1 points = ['A', 'B', 'C', 'D']
2 dist = {(a, b): (0 if a == b else 1)
3         for a in points for b in points}
4 fms = FiniteMetricSpace(carrier=tuple(points), distance=dist)
5 print("is_complete:", is_complete(fms).status)
6 print("is_totally_bounded:", is_totally_bounded(fms).status)
7 mc = metric_compactness_check(fms)
8 print("metric_compactness:", mc.status, mc.value)

```

```
is_complete: true
is_totally_bounded: true
metric_compactness: true True
```

Örnek 5.4 — analyze_metric_completeness

```
1 r = analyze_metric_completeness(fms)
2 print("status:", r.status)
3 print("value:", r.value)
```

```
status: true
value: {'is_complete': True, 'is_totally_bounded': True, 'metric_compact': True}
```

8.6 Alıştırmalar

Kodlama

- K1. Rasyonel sayılar $\mathbb{Q}$ (sembolik) için `is_complete` sonucunu kontrol edin.
- K2. Kendi 4-noktalı metrik uzayınızı oluşturun ve `metric_compactness_check` uygulayın.
- K3. `analyze_metric_completeness` çalıştırın ve `value` sözlüğünü inceleyin.

Teori

- T1. Banach sabit-nokta teoremini sözlü olarak açıklayın ve bir uygulama verin.
- T2. Kapalı $[0, 1]$ 'in tam, açık $(0, 1)$ 'in tam olmadığını gösterin. (Hint: $1/n$ dizisi Cauchy ama $0 \notin (0, 1)$.)

Bölüm 9

Metrik Fonksiyonlar ve Sözleşmeler

Bu bölümde metrik uzaylar arasındaki fonksiyon türleri (izometri, Lipschitz, sürekli) ve bunların sınıflandırılması incelenmektedir.

9.1 Konu

9.1.1 Metrik Fonksiyon Sınıfları

$f : (M_1, d_1) \rightarrow (M_2, d_2)$ fonksiyonu:

Sınıf	Koşul
Genişlemez (non-expansive)	$d_2(f(x), f(y)) \leq d_1(x, y)$
Lipschitz	$\exists K > 0 : d_2(f(x), f(y)) \leq K \cdot d_1(x, y)$
Üniform sürekli	$\forall \varepsilon > 0 \exists \delta > 0 : d_1(x, y) < \delta \Rightarrow d_2(f(x), f(y)) < \varepsilon$
Sürekli	Her noktada ε - δ sürekliliği
İzometri	$d_2(f(x), f(y)) = d_1(x, y)$
Benzerlik	$\exists c > 0 : d_2(f(x), f(y)) = c \cdot d_1(x, y)$
Homeomorfizma	Bijektif sürekli; ters de sürekli

Tablo 9.1: Metrik fonksiyon sınıfları

Sıralama: İzometri $\Rightarrow$ Genişlemez $\Rightarrow$ Lipschitz $\Rightarrow$ Üniform sürekli $\Rightarrow$ Sürekli.

9.1.2 Lipschitz Sabiti

$$K = \sup_{x \neq y} \frac{d_2(f(x), f(y))}{d_1(x, y)}.$$

İzometri: $K = 1$ ve eşitlik; benzerlik: $K = c$ sabit.

9.2 Teoremler

Teorem 9.1. *İzometri $\Rightarrow$ homeomorfizma.*

Teorem 9.2. *Benzerlik, $c = 1 \Rightarrow$ İzometri.*

Teorem 9.3. *Lipschitz $\Rightarrow$ Üniform sürekli $\Rightarrow$ Sürekli.*

Kanıt İskeleti. $\delta = \varepsilon/K$ seçimi Lipschitz $\Rightarrow$ ünif. sürekli gösterir. $\square$

Teorem 9.4. *Kompakt metrik uzaydan metrik uzaya her sürekli fonksiyon üniform sürekli*

9.3 Algoritmalar

9.3.1 classify_finite_metric_map — $O(|X|^2)$

Algorithm 11 ClassifyMap(X, d_1, Y, d_2, f)

```

1:  $K \leftarrow 0$ ; isometry  $\leftarrow$  True; similarity_ratio  $\leftarrow$  None
2: for each pair  $(x_1, x_2)$  with  $x_1 \neq x_2$  do
3:   ratio  $\leftarrow d_2(f(x_1), f(x_2))/d_1(x_1, x_2)$ 
4:    $K \leftarrow \max(K, \text{ratio})$ 
5:   if  $d_2(f(x_1), f(x_2)) \neq d_1(x_1, x_2)$  then
6:     isometry  $\leftarrow$  False
7:   end if
8:   if similarity_ratio is None then
9:     similarity_ratio  $\leftarrow$  ratio
10:  else if ratio  $\neq$  similarity_ratio then
11:    similarity_ratio  $\leftarrow$  None
12:  end if
13: end for
14: return MetricMapProfile(lipschitz_constant =  $K$ , isometry = isometry, ...)
```

Karmaşıklık: $O(|X|^2)$.

9.4 pytop API

```

1 from pytop.metric_spaces import FiniteMetricSpace
2 from pytop.metric_map_taxonomy import (
3     classify_finite_metric_map,
4     MetricMapProfile,
5 )
```

classify_finite_metric_map(fms1, fms2, f) $\rightarrow$ MetricMapProfile döner (Result değil).

MetricMapProfile alanları: isometry, similarity, similarity_ratio, lipschitz_constant, non_expansive, bijective, homeomorphism.

9.5 Örnekler

Örnek 5.1 — Özdeşlik: İzometri

```

1 f_id = {1: 1, 2: 2, 3: 3, 4: 4}
2 prof_id = classify_finite_metric_map(fms, fms, f_id)
3 print("isometry:", prof_id.isometry)
4 print("lipschitz_constant:", prof_id.lipschitz_constant)

```

```

isometry: True
lipschitz_constant: 1.0

```

Örnek 5.2 — $2\times$ Ölçekleme: Benzerlik

```

1 f_scale = {1: 2, 2: 4, 3: 6, 4: 8}
2 prof_scale = classify_finite_metric_map(fms_line1, fms_line2, f_scale
3 )
4 print("similarity:", prof_scale.similarity)
5 print("similarity_ratio:", prof_scale.similarity_ratio)
6 print("lipschitz_constant:", prof_scale.lipschitz_constant)

```

```

similarity: True
similarity_ratio: 2.0
lipschitz_constant: 2.0

```

Örnek 5.3 — Sabit Fonksiyon: $K = 0$

```

1 f_const = {1: 1, 2: 1, 3: 1, 4: 1}
2 prof_const = classify_finite_metric_map(fms, fms, f_const)
3 print("lipschitz_constant:", prof_const.lipschitz_constant)
4 print("bijective:", prof_const.bijective)

```

```

lipschitz_constant: 0.0
bijective: False

```

9.6 Alıştırmalar

Kodlama

- K1. $\{1, 2, 3\}$ üzerinde $f(1) = 2, f(2) = 3, f(3) = 1$ (döngü) fonksiyonunu ayrık metrikte `classify_finite_metric_map` ile sınıflandırın.
- K2. Öklid metriklili $\{0, 1, 2, 3, 4\}$ üzerinde $f(x) = x+1$ kaydırma fonksiyonunun Lipschitz sabitini hesaplayın.
- K3. Sabit fonksiyon $f(x) = 1$ için $K = 0$ olduğunu doğrulayın.

Teori

- T1. İzometri $\Rightarrow$ genişlemez $\Rightarrow$ Lipschitz ($K = 1$) zincirini ispatlayın.
- T2. Lipschitz $\Rightarrow$ üniform sürekli olduğunu $\delta = \varepsilon/K$ seçimi ile ispatlayın.

Dizin

Açık top Bkz. Bölüm 7.

Banach sabit-nokta teoremi Bkz. Bölüm 8.

Bağlantılı uzay Bkz. Bölüm 5.

Cauchy dizisi Bkz. Bölüm 8.

Heine-Borel teoremi Bkz. Bölümler 4, 8.

Homeomorfizma Bkz. Bölümler 1, 9.

İzometri Bkz. Bölüm 9.

Kompaktlık Bkz. Bölüm 4.

Lipschitz sabit Bkz. Bölüm 9.

Metrik aksiyomları Bkz. Bölüm 7.

Sayılabilirlik aksiyomları Bkz. Bölüm 6.

Tamlık Bkz. Bölüm 8.

Topoloji aksiyomları Bkz. Bölüm 1.

Urysohn fonksiyon teoremi Bkz. Bölüm 3.